

- Браузеры используют собственные встроенные стили для форматирования различных элементов. Заголовки обычно отображаются крупным полужирным шрифтом, ссылки — синим цветом и т. д. Даже если определен конкретный размер кегля для текстового контента веб-страницы и применен к элементу `body`, заголовки будут отображены большим по размеру шрифтом. Элементы `h1` будут крупнее `h2`. Точно так же, когда вы устанавливаете цвет шрифта применительно к `body`, гиперссылки веб-страницы все равно будут отображены синим цветом с подчеркиванием.

ПРИМЕЧАНИЕ

Рекомендуется игнорировать встроенные стили браузеров — это упростит создание сайтов, совместимых с различными типами браузеров. В главе 5 вы узнаете, как добиться этого.

Если возникает конфликт, то побеждает более специфичный стиль. Другими словами, когда вы применяете CSS-свойство к элементу веб-страницы (например, устанавливаете размер шрифта для маркированного списка (элемента `ul`)) и оно конфликтует с наследуемым (например, размером шрифта `body`), браузер использует специфичное свойство, более близко относящееся к форматируемому элементу (в данном случае применяется размер шрифта, определенный для `ul`).

ПРИМЕЧАНИЕ

Такие типы конфликтов между стилями встречаются очень часто, и правила, определяющие, как должен вести себя браузер, называются каскадностью. Подробнее об этом вы узнаете в следующей главе.

Практикум: наследование

В этом практикуме, состоящем из трех частей, вы увидите, как функционирует механизм наследования. Сначала создадим простой селектор тега и понаблюдаем, каким образом он передает свои настройки вложенным элементам. Создадим класс, который использует наследование для изменения форматирования всей веб-страницы. И наконец, рассмотрим случаи отступления от правил, на которые следует обратить внимание.

Перед началом урока нужно загрузить архив с файлами примеров, расположенный по адресу github.com/mrightman/css_4e. Перейдите по ссылке и загрузите ZIP-архив с файлами. Файлы текущего практикума находятся в папке 04.

Одноуровневое наследование

Для того чтобы увидеть и понять, как работает механизм наследования, добавим стиль к определенному элементу и посмотрим, как он воздействует на вложенные элементы. Все три части этого практикума взаимосвязаны, поэтому сохраняйте свой файл в конце каждого урока для его последующего использования.

1. Откройте файл `inheritance.html` в редакторе HTML-кода.

Файл уже содержит внутреннюю таблицу стилей с одним селектором тега, придающим элементу `body` фоновый цвет.

ПРИМЕЧАНИЕ

Вообще, в сайтах лучше использовать внешние таблицы стилей по причинам, описанным в главе 2. Иногда проще начать разработку CSS-дизайна отдельных веб-страниц, используя внутреннюю таблицу, как в этом примере, а уже потом преобразовать ее во внешнюю.

2. Добавьте еще один стиль после стиля `<body>`:

```
p {  
  color: rgb(50,122,167);  
}
```

С этим свойством мы работали в практикуме предыдущей главы. Оно устанавливает цвет текста. А ваша таблица стилей уже готова.

3. Чтобы посмотреть на результат работы, откройте страницу в браузере.

Цвет всех четырех абзацев страницы изменился с черного на синий (рис. 4.3).

Обратите внимание, как этот стиль `p` воздействует на *другие* элементы. Они вложены в `p` и также меняют цвет. Например, текст, *заключенный* в `em` и `strong` внутри каждого абзаца, также изменяется на синий, при этом сохраняется выделение полужирным и курсивным начертаниями. В конечном счете устанавливается тот цвет текста абзаца, который вы хотели, *независимо* от любых других элементов.

Без наследования таблицы стилей было бы очень трудно создавать: элементы `em`, `a` и `strong` не унаследовали бы свойство цвета от `p`. Следовательно, пришлось бы создавать дополнительные стили с селекторами потомков, например `p em` или `p strong`, чтобы правильно отформатировать текст.

Но вы увидите, что ссылка в конце первого абзаца не изменила свой цвет, оставшись, как ей и положено, синей. Как уже упоминалось, для определенных элементов у браузеров есть собственные стили, поэтому наследование к ним не применяется. Дополнительные сведения о подобном поведении можно найти в главе 5.

Наследование при форматировании всей веб-страницы

Наследование работает и с классами. Любой элемент с примененным к нему стилем переносит CSS-свойства и на его потомков. Учитывая это, можно пользоваться наследованием для быстрого изменения дизайна всей веб-страницы.

1. Вернитесь к HTML-редактору с открытым файлом `inheritance.html`.

Сейчас мы добавим новый стиль после только что созданного стиля элемента `p`.

2. Щелкните кнопкой мыши сразу за закрывающей скобкой селектора `p`. Нажмите клавишу **Enter** для создания новой строки и введите `.content {`. Теперь дважды нажмите клавишу **Enter** и укажите закрывающую скобку `}`.

Сейчас мы создадим новый класс для `body`, который окружит остальные элементы на странице.

3. Перейдите к строке между двумя скобками и добавьте к стилю следующие свойства:

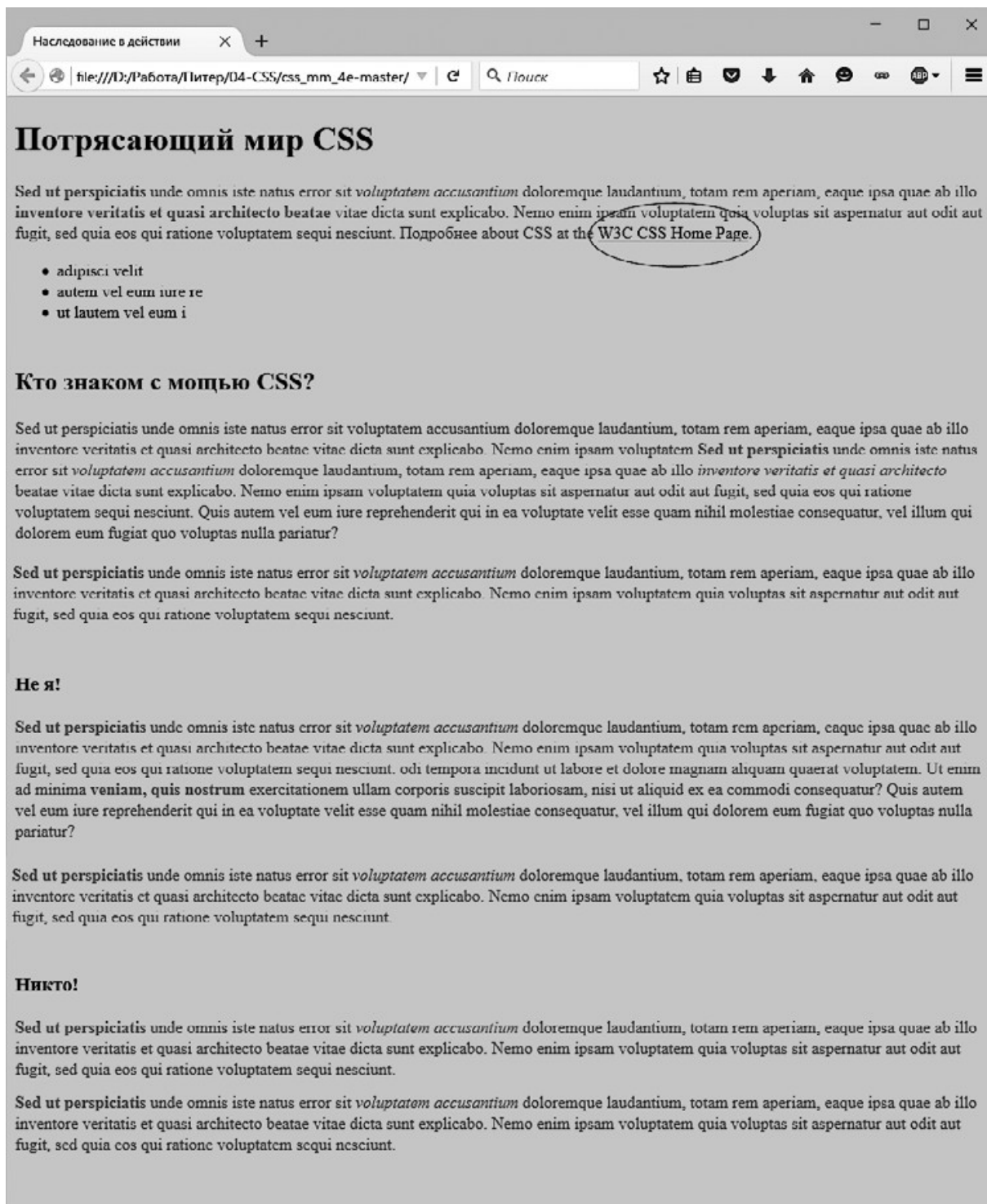


Рис. 4.3. Принцип наследования в действии! Текст, выделенный полужирным шрифтом, приобрел тот же цвет, что и абзац р, окружающий его. Но что это? Ссылка в конце первого абзаца осталась синей (выделена на рисунке). Почему так произошло, вы узнаете в следующей главе

```
font-family: "Helvetica Neue", Arial, Helvetica, sans-serif;
font-size: 18px;
color: rgb(194,91,116);
max-width: 900px;
margin: 0 auto;
```

Законченный стиль должен иметь следующий вид:

```
.content {  
  font-family: "Helvetica Neue", Arial, Helvetica, sans-serif;  
  font-size: 18px;  
  color: rgb(194,91,116);  
  max-width: 900px;  
  margin: 0 auto;  
}
```

Этот класс устанавливает начертание, размер и цвет шрифта. Он также задает ширину и центрирует стиль на странице (мы рассматривали это в практикуме предыдущей главы).

4. Теперь вернитесь к открывающему тегу `<body>` (расположен строкой ниже закрывающего тега `</head>`) и введите перед закрывающей скобкой через пробел `class="content"`.

Теперь тег `<body>` должен выглядеть следующим образом: `<body class="content">`. Благодаря наследованию все элементы, заключенные внутри `body` (текст которых отображен в окне браузера), наследуют свойства стилей и, соответственно, используют тот же шрифт.

5. Сохраните и просмотрите веб-страницу в браузере.

Как видно на рис. 4.4, наш класс обеспечил всему тексту веб-страницы согласованный внешний вид, без резких переходов, плавно сочетающий все фрагменты содержимого. И заголовки, и абзацы, заключенные в `body`, — все элементы веб-страницы за счет изменения свойств шрифта приобрели новый стиль.

Страница выглядит интересно, но теперь рассмотрим ее более детально: изменение цвета затронуло только заголовки и маркированный список на странице, однако текст заголовка имеет другой размер по сравнению с абзацами, даже несмотря на то, что стиль определяет точный размер шрифта. Каким образом каскадные таблицы стилей узнали, что размер заголовка не должен быть таким же, как размер текста? И почему не применили к вложенным в `body` элементам `p` новый цвет?

ПРИМЕЧАНИЕ

Почему мы используем класс `content` вместо стиля элемента `body`, чтобы изменить вид страницы? В данном примере селектор тега еще хорошо работает. Но применение класса к элементу `body` — это отличный способ настроить по индивидуальному образцу внешний вид нескольких страниц сайта. Например, если они все используют одну и ту же таблицу стилей, то стиль элемента `body` будет применяться к `body` на каждой странице вашего сайта. А создавая различные классы (или идентификаторы), вы можете создавать различные стили элемента `body` для разных разделов сайта или разных типов страниц.

Вы видите, в чем суть каскадности таблиц стилей? В этом примере для элемента `p` образовался конфликт стилей, в частности двух одинаковых атрибутов цвета, — стиля элемента, созданного в шаге 2 в подразделе «Одноуровневое наследование» выше, и класса, созданного только что. Если произошла такая ситуация, то браузер должен выбрать один из стилей. Используется более близкий (специфичный) к элементу стиль, то есть цвет, который вы явно назначили `p`. О правилах каскадности будет рассказано в следующей главе.

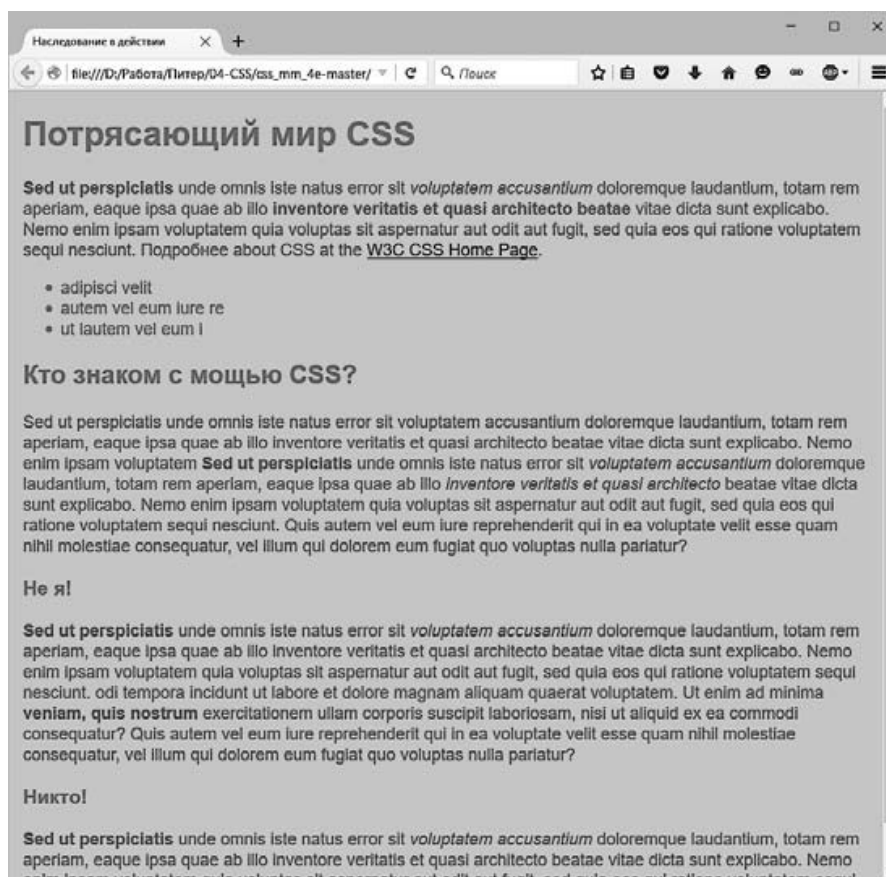


Рис. 4.4. Стиль, примененный к элементу `body`, переносит его свойства на все элементы, отображаемые в окне браузера. Благодаря этому очень легко форматировать эффекты на странице

Исключения механизма наследования

Наследование применяется не всегда, и в некоторых случаях это очень хорошо. Для отдельных свойств наследование имело бы исключительно негативное влияние на дизайн веб-страницы. В заключительной части практикума будет приведен пример *исключения (бездействия)* механизма наследования. Вы увидите, что отступы, поля и границы (среди других свойств) не наследуются вложенными элементами. Далее вы узнаете, почему так предусмотрено.

1. Вернитесь в редактор HTML-кода с открытым файлом `inheritance.html`.
Дополним только что созданный стиль `p`.
2. Найдите определение стиля `p`, щелкните кнопкой мыши сразу за свойством цвета (`color: rgb(50,122,167);`) и нажмите клавишу **Enter** для перехода на новую строку.
Сейчас мы создадим отступ слева для всех абзацев веб-страницы.
3. Добавьте два свойства к стилю следующим образом:

```
p {
  color: rgb(50,122,167);
  padding-left: 20px;
  border-left: solid 25px rgba(255,255,255,.5);
}
```

Этот код изменит отступ слева каждого абзаца и сместит текст таким образом, чтобы он не касался границы. Свойство `padding` создает отступ от границы размером 20 пикселей.

4. Сохраните файл и просмотрите его в браузере.

Обратите внимание, что все абзацы `p` приобрели слева толстую светлую границу. Однако у элементов, *вложенных* в абзац `p` (например, `em`), нет такого отступа или границы (рис. 4.5). Это поведение браузера оправданно: веб-страница выглядела бы странно, если бы каждый элемент `em` и `strong` в абзаце имел дополнительную границу и отступ слева размером 20 пикселей!

Если вы хотите увидеть, что бы случилось, если бы эти свойства наследовались, отредактируйте селектор `p` таким образом: `p, p *`, что сделает его групповым. Первая часть — это тот селектор `p`, который вы только что создали. А вторая часть — `p *` — буквально означает следующее: «выберите все элементы внутри абзаца `p` и примените к ним стиль» (`*` — универсальный селектор, был описан в предыдущей главе).

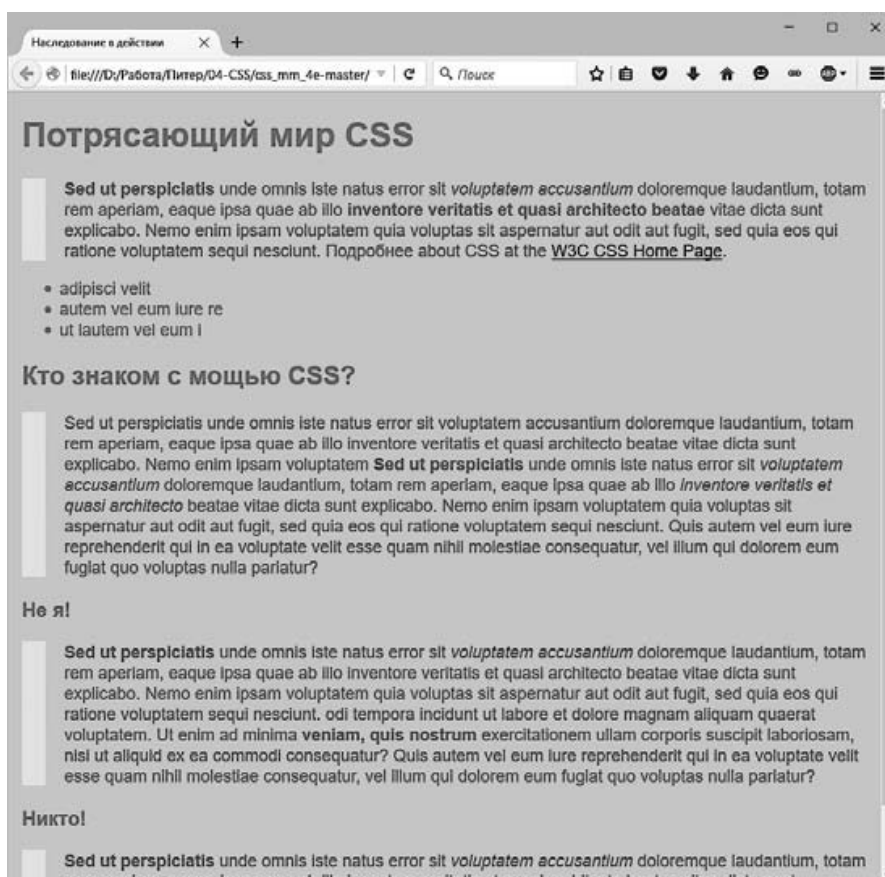


Рис 4.5. Большинство CSS-свойств наследуются вложенными элементами (например, цвет шрифта). Однако поля, отступы и границы являются исключением. Справочная информация относительно наследования CSS-свойств приведена в приложении 1

ПРИМЕЧАНИЕ

Финальный вариант веб-страницы, которую вы создали в этом практикуме, можно найти в папке `04_finished`.